

PGR 210 - Natural Language Processing Part

Kristiania University of Applied Sciences

By Huamin Ren

Huamin.ren@kristiania.no

Outline

- 1. Bag-of-words (BoW)
- 2. Vectorizing
- 3. TF-IDF
- 4. Word embeddings and visualization

1. Bag-of-words

Next step: turn words into continuous numbers rather than just integers to capture the *importance*

- 1.1 Bags of words—Vectors of word counts or frequencies
- 1.2 Bags of n-grams—Counts of word pairs (bigrams), triplets (trigrams), and so on

1.1 Bags of words

- Think for a moment: how to turn 'binary' value to a continuous value?

Input string: " The faster Harry got to the store, the faster Harry, the faster, would get home. "

Divide into words: ['the', 'faster', 'harry', 'got', 'to', 'the', 'store', ',', 'the', 'faster', 'harry', ',', 'the', 'faster', ',', 'would', 'get', 'home', '.']

Count the frequency per each word: {'the': 4, 'faster': 3, 'harry': 2, 'got': 1, 'to': 1, 'store': 1, ',': 3, 'would': 1, 'get': 1, 'home': 1, '.': 1}

Find important words: [('the', 4), ('faster', 3), (',', 3), ('harry', 2)]

Improved one: [('faster', 3), (',', 3), ('harry', 2), ('got', 1)]

skolen
tania

1.1 Bags of words

A ***collections.Counter*** object is an unordered collection, also called a bag or multiset. Depending on your platform and Python version, you may find that a Counter is displayed in a seemingly reasonable order, like lexical order or the order that tokens appeared in your statement. But just as for a standard Python dict, you cannot rely on the order of your tokens (keys) in a Counter.

1.1 Bags of words

- The simplest vector space representational model for unstructured text?
- The Bag of Words model represents each text document as a numeric vector - each dimension is a specific word from the corpus and the value could be its frequency in the document, occurrence (denoted by 1 or 0), or even weighted values.

Outline

- 1. Bag-of-words (BoW)
- **2. *Vectorizing***
- 3. TF-IDF
- 4. Topic models

- Instead of raw word counts to describe your documents in a corpus, use normalized term frequencies.

For example, calculate each word and get the relative importance to the document of that term.

2. Vectorizing

```
Counter({'kite': 16,  
        'traditionally': 1,  
        'tethered': 2,  
        'heavier-than-air': 1,  
        'craft': 2,  
        'wing': 5,  
        'surfaces': 1,  
        'react': 1,  
        'air': 2,  
        'create': 1,  
        'lift': 4,  
        'drag.': 1,  
        'consists': 2,  
        'wings': 1,  
        ',': 15,  
        'tethers': 2,  
        'anchors.': 2,  
        'kites': 8,  
        'often': 2
```

```
[OrderedDict([(' ', 0.05555555555555555),  
              ('.', 0.05555555555555555),  
              ('and', 0.05555555555555555),  
              ('as', 0),  
              ('faster', 0.16666666666666666),  
              ('get', 0.05555555555555555),  
              ('got', 0.05555555555555555),  
              ('hairy', 0),  
              ('harry', 0.05555555555555555),  
              ('harrywould', 0.05555555555555555),  
              ('home', 0.05555555555555555),  
              ('is', 0),  
              ('jill', 0),  
              ('not', 0),  
              ('store', 0.05555555555555555),  
              ('than', 0),  
              ('the', 0.16666666666666666),  
              ('to', 0.05555555555555555)])]
```

2. Vectorizing

- If you're going to do math on these **two vectors**, they need to represent a position in a common space, relative to something consistent!

The first step: to normalize the counts by calculating, and then normalize term frequency.



The second step: make all the vectors of standard length or dimension

Outline

- 1. Bag-of-words (BoW)
- 2. Vectorizing
- **3. *TF-IDF***
- 4. Topic models

3. TF-IDF

- Stands for term frequency times inverse document frequency
 - Term frequencies are the counts of each word in a document
 - Inverse document frequency means that

TF-IDF

(0, 5)	1
(0, 9)	1
(0, 10)	1
(0, 7)	1
(1, 6)	1
(1, 2)	1
(1, 1)	1
(1, 12)	1
(1, 3)	1
(1, 8)	1
(1, 0)	1
(1, 4)	1
(2, 5)	1
(2, 9)	1
(2, 10)	1
(2, 7)	1
(2, 11)	1

[	[	0	0	0	0	0	1	0	1	0	1	1	0	0]
[	1	1	1	1	1	0	1	0	1	0	0	0	1]	
[	0	0	0	0	0	1	0	1	0	1	1	1	0]]	

corpus = ['sentence1: similar to sentence3','sentence2: do'
'not care what other sentences are saying','sentence3:
very similar to sentence1']



['are', 'care', 'donot', 'other', 'saying', 'sentence1', 'sentence2',
'sentence3', 'sentences', 'similar', 'to', 'very', 'what']

2. TF-IDF

$$tf(w, D) = f_{w_D}$$

where f_{w_D} denoted frequency for word $\mathbf{w}$ in document $\mathbf{D}$, which becomes the term frequency (tf).

- TF: term frequency
- IDF: inverse document frequency

More thoughts

- By looking at an even more useful vector representation that counts the number of occurrences, or frequency, of each word in the given text, you assume that the more times a word occurs, the more meaning it must contribute to that document.
- Or if you have classified some words as expressing positive emotions—words like “good,” “best,” “joy,” and “fantastic”—the more a document that contains those words is likely to have positive “sentiment.”

Even more thoughts

- Let's say you find the word "dog" 3 times in document A and 100 times in document B. Clearly "dog" is way more important to document B while using TF.
- Wait!

$$TF(\text{"dog," } document_A) = 3/30 = .1$$

$$TF(\text{"dog," } document_B) = 100/580000 = .00017$$

- Instead of raw word counts to describe your documents in a corpus, use normalized term frequencies.

For example, calculate each word and get the relative importance to the document of that term.

TF-IDF

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

$\text{idf}(w, D)$ represents the idf for the term/word w in document D , N represents the total number of documents in our corpus, and $\text{df}(t)$ represents the number of documents in which the term w is present.

```
[[0.      0.      0.      0.      0.      0.5
  0.      0.5      0.      0.5      0.5      0.
  0.      ]
 [0.35355339 0.35355339 0.35355339 0.35355339 0.35355339 0.
  0.35355339 0.      0.35355339 0.      0.      0.
  0.35355339]
 [0.      0.      0.      0.      0.      0.41779577
  0.      0.41779577 0.      0.41779577 0.41779577 0.54935123
  0.      ]]
```

- Suppose you have a corpus of 1 million documents, someone searches for the word “cat,” and in your 1 million documents, you have exactly 1 document that contains the word “cat.” The raw IDF of this is

$$1,000,000 / 1 = 1,000,000$$

- Suppose you have 10 documents with the word “dog” in them, the IDF for “dog” is:

$$1,000,000 / 10 = 100,000$$

Similarly you can also implement IDF

- Be noted:

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

$$\text{tf}(t, d) = \frac{\text{count}(t)}{\text{count}(d)}$$

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) * \text{idf}(t, D)$$

- TF-IDF for a document: K-dimensional vector representation of each document in the corpus.
- Calculate the similarity: two vectors are considered similar if their cosine similarity is high, so minimize:

$$\cos \Theta = \frac{A \cdot B}{|A| |B|}$$

Two vectors, in a given vector space, can be said to be similar if they have a similar angle. If you imagine each vector starting at the origin and reaching out its prescribed distance and direction, the ones that reach out at the same angle are similar, even if they don't reach out to the same distance.

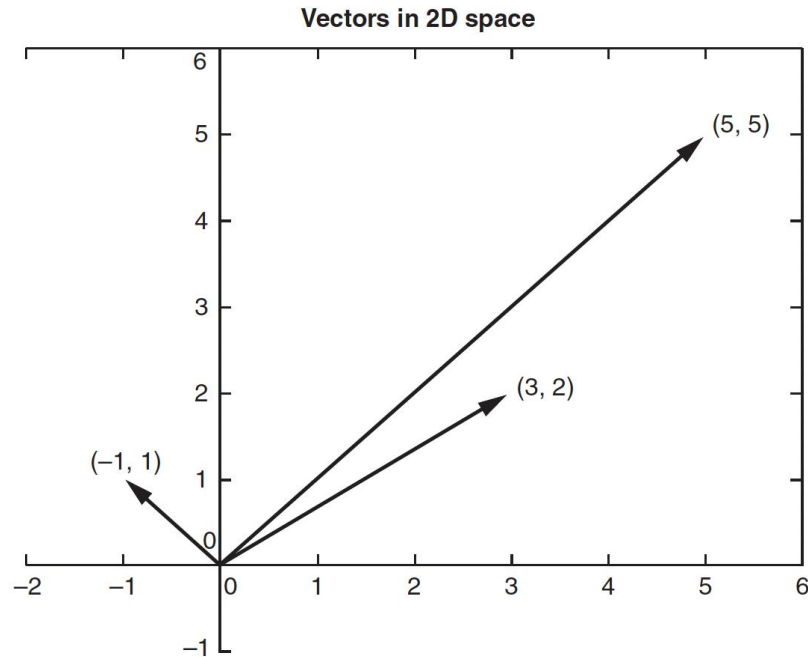
- The more times a word appears in the document, the TF (and hence the TF-IDF) will go up
- The number of documents that contain that word goes up, the IDF (and hence the TF-IDF) for that word will go down
- In some classes, all the calculations will be done in log space so that multiplications become additions and division becomes subtraction

Similarity?

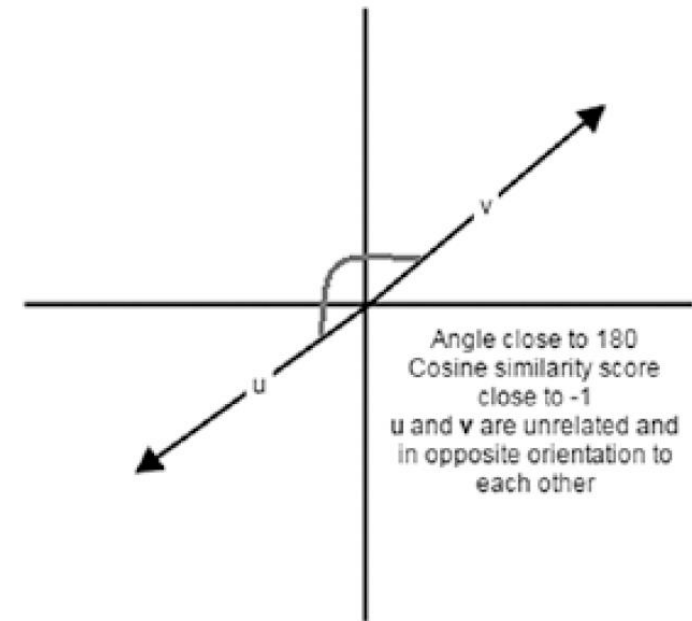
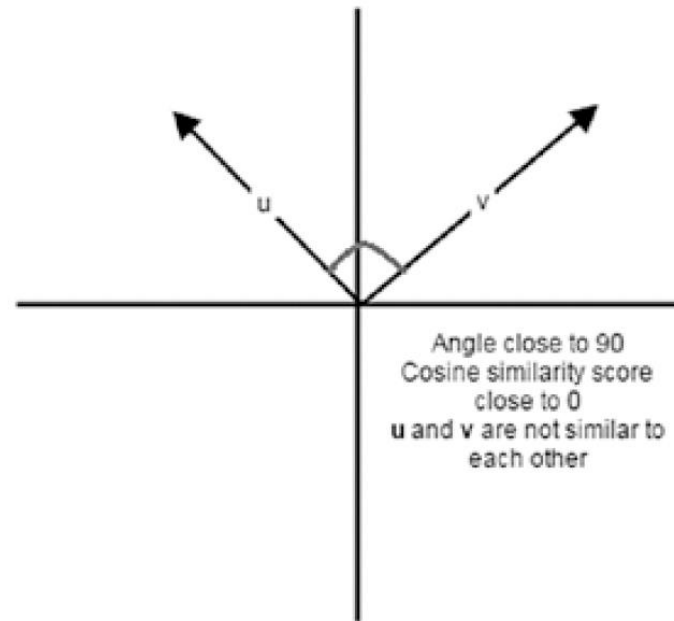
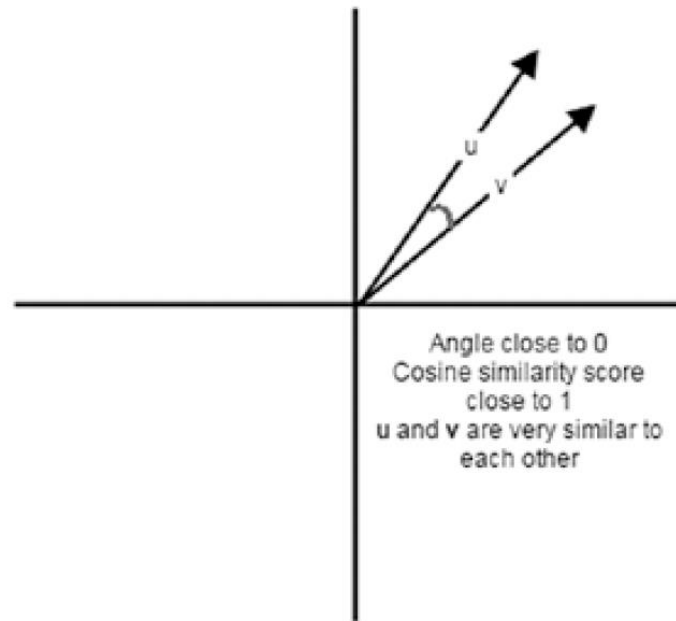
- Document similarity is the process of using a distance or similarity based metric that can identify how similar a text document is to any other document(s) based on features extracted from the documents, like Bag of Words (BoW) or TF-IDF.

- Pairwise document similarity in a corpus involves computing document similarity for each pair of documents in a corpus.
- Thus, if you have C documents in a corpus, you would end up with a $C \times C$ matrix, such that each row and column represents the similarity score for a pair of documents.

	0	1	2
0	1.0000000	0.0	0.071144
1	0.0000000	1.0	0.0000000
2	0.071144	0.0	1.0000000



- Cosine similarity gives us a metric representing the cosine of the angle between the feature vector representations of two text documents.
- The smaller the angle between the documents, the closer and more similar they are.



Outline

- 1. Bag-of-words (BoW)
- 2. Vectorizing
- 3. TF-IDF
- 4. Word embeddings and visualization

4.1 Understand vectorization

- Task: map each word in the vocabulary (V) of the text corpus to a unique ID (integer value), then represent each sentence or document in the corpus as a V-dimensional vector.

D1 Dog bites man.

D2 Man bites dog.

D3 Dog eats meat.

D4 Man eats food.

4.1 Understand vectorization

- One-hot encoding
 - In one-hot encoding, each word w in the corpus vocabulary is given a unique integer ID w_{id} that is between 1 and $|V|$, where V is the set of the corpus vocabulary.
 - Each word is then represented by a V -dimensional binary vector of 0s and 1s. This is done via a $|V|$ dimension vector filled with all 0s barring the index, where index = w_{id} .

4.1 Understand vectorization

D1 Dog bites man.

D2 Man bites dog.

D3 Dog eats meat.

D4 Man eats food.

dog = 1, bites = 2, man = 3, meat = 4, food = 5, eats = 6.

Dog is represented as [1 0 0 0 0 0], as the word “dog” is mapped to ID 1.

Bites: [0 1 0 0 0 0]

...

D1: [[1 0 0 0 0 0] [0 1 0 0 0 0] [0 0 1 0 0 0]]

D2...

D4: [[0 0 1 0 0] [0 0 0 0 1 0] [0 0 0 0 0 1]]

4.1 Understand vectorization

- The size of a one-hot vector is directly proportional to size of the vocabulary
- This representation does not give a fixed-length representation for te
- Out of vocabulary (OOV) problem: “man eats fruits.” , however, training data didn’t include “fruit” and there’s no way to represent it in our model.

4.2 Visualizing Embeddings - t-SNE

- T-SNE
 - Also called *t-distributed Stochastic Neighboring Embedding*
 - Visualizing high-dimensional data like embeddings by reducing them to two- or three-dimensional data
- We need to get back to this later this course – need more knowledge to be able to use t-SNE 😊